RPS ARC-AGI-3 Solutions Technical Report

Jason Feng
https://x.com/iamjasonfeng
August 9, 2026

Abstract

> This paper documents three of my ARC-AGI-3 solutions: Gorilla-1.1, Sandwich, and Tiger. The solutions are based on Regressive Plasticity Schedule (RPS), a post-training method I created. Gorilla-1.1 applies RPS during post-training through a three-stage multimodal DPO curriculum with lower learning rates in later stages. Sandwich applies RPS at test time through an Explore-to-Stable proposal tournament. Tiger applies RPS at test time to within-level working memory, cross-level memory, and occasional surprise proposals. I built all three solutions on the Tufa Labs Duck harness and used Qwen3.6-27B as their underlying model. I present this as a technical methods report: I document the approaches and released artifacts, but I do not claim that any method is superior to Tufa, to another existing system, or to either of the other methods described here. ARC-AGI-3 agent trajectories are stochastic, and as of this writing I have not been able to perform enough independent competition runs under the one-submission-per-day limit to support a reliable quantitative comparison. Evaluation is ongoing, and readers can visit the linked Kaggle notebooks, where Kaggle automatically displays each notebook's best competition score.

Update: August 10, 2026

I identified a serious flaw in Gorilla-1.1 Stage 2C: all 166 DPO pairs used a shared prompt that described the interaction as an "offline rejected-candidate call," including the chosen branches. Sandwich and Tiger were not affected.

1. Introduction

ARC-AGI-3 is an interactive reasoning benchmark in which agents must explore unfamiliar environments, infer goals and mechanics, and act efficiently.[1]

Regressive Plasticity Schedule (RPS) is a curriculum principle in which a model first learns easier or more foundational material with higher plasticity, then works on harder material with lower plasticity.[2] In post-training, the learning rate provides a direct control over plasticity. At test time, no weights are updated, so plasticity must be represented behaviorally: through how freely an agent changes hypotheses, proposals, plans, or memory.

I describe three attempts to carry that principle into ARC-AGI-3:

- **Gorilla-1.1** uses a training-time RPS curriculum. A Qwen3.6-27B model[3] is trained with multimodal DPO over easier ARC Witness data,[4] harder ARC Witness data, and additional hard ARC Interactive data[5] while the learning rate decreases across stages.
- **Sandwich** uses a test-time proposal schedule. Explore proposals permit broader hypothesis generation; Stable proposals must refine evidence-supported mechanics.
- **Tiger** uses test-time memory schedules. It separately controls within-level working memory and persistent cross-level memory, while a symbolic controller occasionally introduces surprising multi-action proposals.

My purpose is documentation, not a leaderboard claim. I explain what I built, how the three approaches relate to RPS, what artifacts I have released, and what limitations remain.

2. Background

2.1 The Original RPS Experiment on ARC-AGI-1 and ARC-AGI-2

In my original RPS paper, I tested a two-stage post-training schedule on Qwen3-8B using managed Direct Preference Optimization (DPO) fine-tuning with LoRA.[6][2] I trained Stage 1 on 400 easier ARC-style preference pairs with a base learning rate of `1e-5`. I trained Stage 2 on 600 harder ARC-AGI-2 preference pairs with `1e-6`, or 10% of the Stage 1 learning rate. For the Equal Plasticity Schedule (EPS) control, I used the same Stage 1 checkpoint, same Stage 2 data, and same within-stage cosine schedule, but retained a Stage 2 base learning rate of `1e-5`.

On the ARC-AGI-1 public evaluation, RPS produced 17 correct test outputs out of 419, compared with 10/419 for EPS. At the task level, RPS solved 14/400 tasks and EPS solved 10/400. RPS also produced more executable programs: 1,145/1,200 attempts executed without error, compared with 870/1,200 for EPS.

Neither model solved an ARC-AGI-2 public-evaluation test output; both scored 0/167. However, RPS produced executable programs on 234/240 attempts, compared with 188/240 for EPS. I interpreted these results as preliminary evidence that coupling a curriculum transition to lower plasticity can improve ARC-style behavior and program-synthesis reliability. I also emphasized that the experiment did not solve ARC-AGI-2 and did not establish broad generalization.

2.2 From Training-Time RPS to Test-Time RPS

In the original experiment, I operationalized plasticity through learning rate. Sandwich and Tiger do not update weights during evaluation, so I treat their use of RPS as a test-time implementation rather than a post-training schedule.

I use **test-time plasticity** to mean the amount of freedom an agent has to revise its current policy-like state. Depending on the solution, that state may be a set of candidate actions, a working interpretation of the current level, or persistent memory carried across levels. A high-plasticity phase encourages wider changes and competing hypotheses. A Stable phase preserves useful structure and permits narrower, evidence-backed refinement.

I use this translation as a practical analogy. I do not claim that prompt-level or symbolic state control is identical to changing neural plasticity through optimization.

2.3 Shared Tufa Foundation

I built all three solutions on the open-source Tufa Labs Duck harness for ARC-AGI-3.[7] The original solver was written by Harold Bessis, Jeroen Cottaar, Isaiah Pressman, Andries Smit, Michal Tesnar, and Stefano Viel. The harness supplies the interactive game loop, visual observations, Python inspection and action tools, persistent history, concurrency, timeout behavior, benchmark integration, and scoring path.

I implemented the modifications below as notebook-level additions around that foundation. Gorilla-1.1 loads my trained LoRA adapter onto the Qwen3.6-27B evaluation model family. I used the vanilla Qwen3.6-27B FP8 model for Sandwich and Tiger.

3. Gorilla-1.1

3.1 Overview

Gorilla-1.1 is a multimodal Qwen3.6-27B LoRA adapter trained with DPO and an RPS curriculum. Its public notebook, adapter, and dataset are listed in Section 9.

The released evaluation notebook is a clean control around the adapter. It retains Tufa's original system prompt, native current-grid images, inspection calls, multi-action support, persistent tool history, generation settings, temperature, output-token behavior, concurrency, timeouts, benchmark flow, and scoring behavior. It adds no proposal system, memory overlay, search module, or other custom evaluation method.

3.2 Child-Teen-Adult Curriculum

Gorilla-1.1 extends the two-stage RPS schedule into three stages:

Analogy	Stage	Data	Source pairs	Learning rate
Child	Stage 1	easier ARC Witness trajectories	201	3.333333 333 333e-6
Teen	Stage 2	harder ARC Witness trajectories	173	8.333333 333 333e-7
Adult	Stage 2C	hard ARC Interactive trajectories	166	3.333333 333 333e-7

Stage 2 uses 25% of the Stage 1 learning rate. Stage 2C continues on hard data with 10% of the Stage 1 learning rate. The child-teen-adult analogy describes the intended schedule: foundational learning occurs with the most plasticity, difficult specialization begins with less plasticity, and continued difficult training occurs with still less.

3.3 Dataset Construction

The complete Gorilla-1.1 dataset contains 540 source-level multimodal DPO pairs in curriculum order. It contains 8,168 chosen action rounds and 8,168 native 512x512 pre-action frames. No image-to-text conversion or context compaction is used.

Stage 1 and Stage 2 chosen branches use Kimi K3[8] ARC Witness chain-of-thought (CoT) filling.[9] Their rejected branches use Qwen3-8B source-level attempted trajectories. Stage 2C chosen branches are complete, replay-verified ARC Interactive level trajectories with Kimi K3 CoT filling. Stage 2C rejected branches use Qwen3-VL-8B-Instruct responses. Chosen and rejected prompts are identical within every pair.

Rows are stored as complete level trajectories. During training preparation, examples that exceed the branch-length limit are split only at complete chosen-action boundaries. The pipeline does not truncate tokens or split inside a reasoning and tool-call round.

CoT filling differs from ordinary solving data generation: the teacher generates reasoning around an existing correct action trajectory rather than discovering the trajectory independently. This supplies action-complete multimodal supervision, but it may not capture every failure, recovery, and hypothesis-selection behavior that appears during unconstrained normal solving.

3.4 Training Configuration

All three stages use one epoch and a `constant_with_warmup` schedule with 5% warmup. Training uses:

Base model	Qwen3.6-27B
Method	multimodal DPO with LoRA
Maximum branch length	11,000 tokens
LoRA rank / alpha / dropout	64 / 128 / 0.05
DPO beta	0.1
RPO alpha	1.0
Precision	BF16
Attention	SDPA
Distributed training	ZeRO-3 across two GPUs
Per-device batch size	1
Gradient accumulation	1
Vision transformer	frozen
Aligner	frozen
Image-to-text conversion	none

Stage 2 is initialized from the cumulative Stage 1 adapter. Stage 2C is initialized from the cumulative Stage 2 adapter. The final Gorilla-1.1 adapter therefore contains the complete three-stage curriculum.

3.5 Relationship to RPS

Gorilla-1.1 is my direct training-time implementation of RPS for ARC-AGI-3. I control plasticity through learning rate rather than through a test-time symbolic state. I increase difficulty from Stage 1 to Stage 2, while Stage 2C keeps the hard-data regime and reduces plasticity further.

4. Sandwich

4.1 Overview

Sandwich is a test-time RPS proposal tournament. The main Tufa action agent may request six independent advisory proposals before choosing an action. Proposal generation does not update model weights, and proposers cannot execute game actions. The main agent remains responsible for judging the proposals and acting through Tufa's normal Python tool.

4.2 Explore and Stable Proposal Modes

Every level begins in Explore mode. In this mode, proposals may branch into competing causal hypotheses or suggest purposeful information-gathering actions. When the main model judges that the evidence is sufficient, it may make a one-way transition to Stable mode for the rest of the level. Stable proposals must remain within evidence-supported mechanics and focus on execution, robustness, refinement, or contingency planning. A new level resets the proposal state to Explore.

Each consultation always contains six sequential proposal calls:

Mode	Proposal temperatures
Explore	four proposals at `0.8`, two at `0.9`
Stable	two proposals at `0.7`, two at `0.8`, two at `0.9`

The number of proposals and temperatures are fixed by the harness. The mode changes what the proposal model is allowed to do, while the ordinary Tufa action-agent settings remain unchanged.

4.3 Consultation and Action Control

The main agent invokes the proposer through a special intercepted Python consultation request. The consultation returns advisory text to the main model rather than an environment action. Only one consultation may occur during an action turn, and the six proposal calls run sequentially within the game worker.

After receiving the proposals, the main agent is instructed to compare them critically, reject weak advice, synthesize compatible ideas when useful, and then make a normal Tufa Python call. This separation keeps proposal generation outside the environment's action channel.

4.4 Relationship to RPS

Sandwich treats proposal policy as the plastic object. Explore allows broad variation; Stable restricts future proposals to refinement of accumulated evidence. The schedule is reset per level because each level may introduce new mechanics. This is a test-time implementation of RPS: no optimizer or model weights are involved.

5. Tiger

5.1 Overview

Tiger combines four test-time mechanisms while retaining the vanilla Qwen3.6-27B FP8 model:

- Within-Level Explore-Stable (WLES)
- Within-Level Memory RPS (WLMRPS)
- Memory RPS (MRPS)
- Surprise Proposer

Tiger keeps Tufa's ordinary action-call temperature, output-token behavior, timeout, concurrency, deadline handling, tool schema, benchmark loop, and scoring behavior unchanged. The new behavior is installed through notebook-level runtime hooks rather than a modified Tufa source bundle.

5.2 WLES and WLMRPS

WLES lets the model decide when the next call within a level should move from Explore to Stable. WLMRPS applies that state to the current-level working world model, goal model, action model, findings, open questions, and plan.

Every level begins in Explore. During this phase, the model may compare, replace, or reorganize working hypotheses as evidence changes. When it emits the designated transition marker, the next call enters Stable. Stable is one-way until the level ends: the model should preserve its selected interpretation and make only targeted, evidence-backed corrections. The next level resets working memory to Explore. WLES and WLMRPS do not change sampling temperature.

5.3 MRPS Across Levels

MRPS separately controls persistent cross-level memory. Level 1 is the high-plasticity phase. When Level 1 is cleared, the model consolidates its evidence into an operational cross-level policy. Level 2 and all later levels use Stable MRPS: persistent policy should be preserved, with later observations represented as small refinements, corrections, or explicit exceptions.

The separation between WLMRPS and MRPS is important. A model can revise its current-level working theory without automatically rewriting the cross-level policy. Memory remains natural language and is kept compact through prompting rather than symbolic truncation.

5.4 Surprise Proposer

During Explore, a per-game symbolic controller activates Surprise Proposer on 20% of eligible action turns, with a maximum of five activations per level. An activation inserts one Qwen3.6-27B proposal call at temperature `0.9` before the ordinary action call. It returns exactly five proposals:

- two normal proposals;
- three independently hyper-fixated proposals.

Each proposal contains an ordered series of at least two actions, with checkpoints or stopping conditions when useful. The proposer has no tools, takes no action, and writes no memory. The immediately following call returns to normal Tufa settings and receives the proposals once. The main model is told that the preceding call was randomly transformed, must evaluate every proposal seriously, and may dismiss one only for a specific evidence-based reason. Unless all five are rejected for such reasons, it tests the shortest informative prefix of the most promising surviving proposal.

5.5 Relationship to RPS

Tiger applies RPS to two different timescales. WLES and WLMRPS regulate within-level working-memory plasticity; MRPS regulates cross-level persistent-memory plasticity. Surprise Proposer injects occasional diversity only while within-level memory remains in Explore. Unlike Sandwich, Tiger does not use an explicit six-proposal tournament and does not change temperature as part of its memory schedule.

6. Evaluation Status

I do not present ARC-AGI-3 scores or rank Gorilla-1.1, Sandwich, and Tiger. ARC-AGI-3 agent trajectories are stochastic, and a single trajectory can produce a substantially different score from another run of the same notebook. As of August 9, 2026, the competition permits one scored submission per day. I have not completed enough independent competition runs to support a reliable comparison among the methods or against other systems.

Quantitative evaluation is ongoing. I plan to submit these solutions repeatedly as the competition's submission limit permits. Readers can visit the linked Kaggle notebooks, where Kaggle automatically displays each notebook's best competition score.

7. Discussion

I use the three systems to explore different locations for an RPS schedule:

Solution	What changes from high plasticity to Stable	Weight updates during evaluation
Gorilla-1.1	model parameters across curriculum stages	adapter trained before evaluation
Sandwich	proposal policy within each level	no
Tiger	working memory within levels and persistent memory across levels	no

Gorilla-1.1 moves the intervention into post-training and leaves the evaluation harness otherwise unchanged. Sandwich is the narrowest test-time implementation: it changes the type of advice produced by an optional proposal tournament while leaving the main action agent in control. Tiger broadens the test-time idea to memory and introduces a stochastic proposal mechanism.

I do not present these approaches as mutually exclusive or ordered by quality. I treat them as three concrete ways to interpret the same general idea. In future experiments, I may combine a trained RPS adapter with carefully controlled test-time RPS, but I would evaluate such combinations separately because additional calls and memory controls can alter runtime as well as behavior.

8. Limitations

First, I provide no quantitative ARC-AGI-3 comparison because my available submission budget has not supported enough repeated competition evaluations.

Second, I use behavioral analogues of plasticity in Sandwich and Tiger. Their Explore and Stable states are prompt and harness constructs, not measurements of neural plasticity.

Third, I combine several mechanisms at once in Tiger. Because I have not completed the necessary ablations, I cannot isolate the effects of WLES, WLMRPS, MRPS, and Surprise Proposer.

Fourth, my proposal methods add inference calls and can increase runtime. Their usefulness depends on whether the additional deliberation produces better actions before the benchmark deadline.

Fifth, I use CoT-filling chosen data for Gorilla-1.1. Although the trajectories are action-complete and replay-verified, reasoning generated around known actions may differ from reasoning produced while discovering actions under normal solving conditions.

Sixth, I trained Gorilla-1.1 with one model family, one preference-data construction, one curriculum ordering, and one learning-rate schedule. I have not established that the same settings generalize to other models or interactive benchmarks.

Finally, I inherit capabilities and constraints from the Tufa Duck harness in all three solutions. I do not present them as standalone ARC-AGI-3 systems independent of that foundation.

9. Solution Links

Gorilla-1.1

- Evaluation notebook
- Adapter on Hugging Face
- Adapter on Kaggle
- Training dataset

Sandwich

- Evaluation notebook

Tiger

- Evaluation notebook

10. Contributions

I created RPS and designed its training-time and test-time applications described here. I designed the Gorilla-1.1 curriculum, directed its dataset construction and training, and selected its evaluation configuration. For Sandwich and Tiger, I developed the RPS schedules, proposal behaviors, and memory-control designs in collaboration with Codex.

The underlying Duck harness and solver are the work of Tufa Labs. I do not claim the model architectures, teacher models, ARC Witness, ARC Interactive, Kaggle, Hugging Face, or ARC Prize infrastructure as my contributions.

11. Acknowledgments

I created the RPS idea myself. OpenAI Codex[10] assisted me in training Gorilla-1.1. Sandwich and Tiger were co-developed by me and Codex. I thank Tufa Labs for releasing the Duck harness on which these solutions are built.

12. Conclusion

I apply RPS at different points in an ARC-AGI-3 system through Gorilla-1.1, Sandwich, and Tiger. Gorilla-1.1 schedules learning rate across a multimodal DPO curriculum, Sandwich schedules proposal behavior, and Tiger schedules working and persistent memory while adding surprise proposals. I document these mechanisms and their released artifacts without claiming that one solution is superior. I am continuing repeated competition evaluation, and the linked Kaggle notebooks will provide the current best scores displayed by Kaggle.

References

1. ARC Prize Foundation. (2026). ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence.
2. Feng, J. (2026). Regressive Plasticity Schedule: A Two-Stage Post-Training Schedule for ARC Program Synthesis.
3. Qwen Team. (2026). Qwen3.6-27B model card.
4. Guanghan. ARC Witness environments.
5. ARC-Interactive contributors. ARC-Interactive: Community game environments for ARC-AGI-3.
6. Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., and Finn, C. (2023). Direct Preference Optimization: Your Language Model Is Secretly a Reward Model.
7. Tufa Labs. (2026). TAAF Duck Harness Kaggle notebook.
8. Moonshot AI. (2026). Kimi K3: Open Frontier Intelligence.
9. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., and Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.
10. OpenAI. Codex.